

Cau 1:

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
using System.Text;
```

```
using System.Threading.Tasks;
```

```
namespace TrialTest
```

```
{
```

```
    class Employee
```

```
    {
```

```
        public string Name { get; set; }
```

```
        public int Age { get; set; }
```

```
        public double Salary { get; set; }
```

```
        public Employee(string name, int age, double salary)
```

```
        {
```

```
            Name = name;
```

```
            Age = age;
```

```
            Salary = salary;
```

```
        }
```

```
        public void DisplayInfo() => Console.WriteLine($"Employee: {Name},  
Age: {Age}, Salary: {Salary}");
```

```
    }
```

```
}
```

Notification systems:

Folder Entities:

- Notification.cs

```
namespace NotificationSystem.Entities
```

```
{
```

```
    public class Notification
```

```
    {
```

```
        public string Recipient { get; set; }
```

```
        public string Message { get; set; }
```

```
        public Notification(string recipient, string message)
```

```
        {
```

```
            Recipient = recipient;
```

```
            Message = message;
```

```
        }
```

```
    }
```

```
}
```

Folder Interfaces:

INotification.cs:

```
using NotificationSystem.Entities;
```

```
namespace NotificationSystem.Interfaces
```

```
{
```

```
    public interface INotification
```

```
    {
```

```
        void SendNotification(Notification notification);
```

```
    }
```

```
}
```

Folder Notifications:

EmailNotification.cs:

```
using NotificationSystem.Entities;
```

```
using NotificationSystem.Interfaces;
```

```
namespace NotificationSystem.Notifications
```

```
{
```

```
    public class EmailNotification : INotification
```

```
    {
```

```
        // Dùng cho trường hợp không có đối tượng Notification thì mình có thể tự khởi tạo  
        Email và truyền trực tiếp vào
```

```
        private readonly string _recipient;
```

```
        private readonly string _message;
```

```
        public EmailNotification(string recipient, string message)
```

```
    {
```

```

        _recipient = recipient;
        _message = message;
    }

```

// Phương thức này sử dụng với 2 cách là dùng từ đối tượng Notification hoặc là dùng thuộc tính của EmailNotification

```

    public void SendNotification(Notification notification)
    {
        Console.WriteLine($"Send Email to {_recipient} with content:" +
            $" {notification.Message}");
    }

    public void SendNotificationVersion2()
    {
        Console.WriteLine($"Send Email to {_recipient} with content:" +
            $" {_message}");
    }

}
}

```

PushNotification.cs:

```

using NotificationSystem.Entities;
using NotificationSystem.Interfaces;

namespace NotificationSystem.Notifications
{
    public class PushNotification : INotification

```

```

{
    // Dùng cho trường hợp không có đối tượng Notification thì mình có thể tự khởi tạo và
    truyền trực tiếp vào

    private readonly string _recipient;

    public PushNotification(string recipient)
    {
        _recipient = recipient;
    }

    // Phương thức này sử dụng với 2 cách là dùng từ đối tượng Notification hoặc là dùng
    thuộc tính của PushNotification

    public void SendNotification(Notification notification)
    {
        Console.WriteLine($"Push Notification to" +
            $"{_recipient} with content:" +
            $" {notification.Message}");
    }
}

```

SmsNotification.cs:

```

using NotificationSystem.Entities;
using NotificationSystem.Interfaces;

namespace NotificationSystem.Notifications
{
    public class SmsNotification : INotification

```

```

{
    private readonly string _recipient;

    public SmsNotification(string recipient)
    {
        _recipient = recipient;
    }

    // Phương thức này sử dụng với 2 cách là dùng từ đối tượng Notification hoặc là dùng
    thuộc tính của SmsNotification

    public void SendNotification(Notification notification)
    {
        Console.WriteLine($"Send SMS to {_recipient} with content: {notification.Message}");
    }
}

```

Folder services:

NotificationSender.cs:

using NotificationSystem.Entities;

using NotificationSystem.Interfaces;

namespace NotificationSystem.Services

```

{
    public class NotificationSender
    {
        private readonly INotification _notification;
    }
}

```

```

// Constructor để inject đối tượng INotification
public NotificationSender(INotification notification)
{
    _notification = notification;
}

public void Send(Notification notification)
{
    _notification.SendNotification(notification);
}
}

```

Program.cs:

```

using NotificationSystem.Entities;
using NotificationSystem.Interfaces;
using NotificationSystem.Notifications;
using NotificationSystem.Services;

```

```

namespace NotificationSystem

```

```

{

```

```

    class Program

```

```

    {

```

```

        // Lưu ý đây làm theo cách giả lập 1 hệ thống nên xét theo khái niệm Dependencies
        Injection(DI)

```

```

        // chúng ta phải chia ra nhiều class khác nhau nhưng ở đây chỉ dùng 1 class duy nhất là
        Program để mô phỏng nhiều phương thức

```

// Thế nên class NotificationSender phải tạo đủ để thỏa mãn điều kiện mỗi đối tượng dùng các sender riêng biệt. không gồm chung.

```
static void Main(string[] args)
{
    // Tạo đối tượng Notification

    var emailNotification = new Notification("user@example.com", "This is notification by Email.");

    var smsNotification = new Notification("1234567890", "This is notification by SMS.");

    var pushNotification = new Notification("user_device_id", "This is notification by Push Notification.");


    // Inject EmailNotification vào NotificationSender

    INotification emailNotifier = new EmailNotification("user@example.com","Hello CSE 422");

    NotificationSender emailSender = new NotificationSender(emailNotifier);

    emailSender.Send(emailNotification);


    // Inject SmsNotification vào NotificationSender

    INotification smsNotifier = new SmsNotification("1234567890");

    NotificationSender smsSender = new NotificationSender(smsNotifier);

    smsSender.Send(smsNotification);


    // Inject PushNotification vào NotificationSender

    INotification pushNotifier = new PushNotification("user_device_id");

    NotificationSender pushSender = new NotificationSender(pushNotifier);

    pushSender.Send(pushNotification);
```


// Hệ thống dễ dàng mở rộng thêm các loại thông báo mới mà không cần thay đổi mã hiện tại.

```
}  
  
}  
  
}
```

Cau 3: Design systems:

Solution

1. Roles of Each Component

- **Frontend:**
 - **Web App:** Provides a user interface for customers to browse hotels, check availability, and book rooms, and for hotel administrators to manage inventory and bookings.
 - **Mobile App:** Offers similar functionality to the web app, optimized for mobile devices, allowing customers to manage bookings on the go.
- **API Gateway:** Serves as the entry point for all client requests, routing them to appropriate microservices, handling authentication, and optionally aggregating responses.
- **Backend Services (Microservices):**
 - **Authentication & Authorization Service:** Manages user accounts, login, and access control, issuing tokens for secure interactions.
 - **Room Management Service:** Handles room details (availability, pricing, types), providing data for booking displays.
 - **Booking Service:** Processes reservations, cancellations, and updates, coordinating with the Room Management Service.
 - **Payment Service:** Manages payment processing and refunds, integrating with external payment providers.
 - **Notification Service:** Sends confirmations, reminders, and alerts via email, SMS, or push notifications.
- **Load Balancer:** Distributes incoming traffic across service instances to ensure scalability and availability.

- **Firewall (DMZ):** Protects the system from external threats by controlling network traffic.
- **Database:** Each microservice has its own database to maintain independence and loose coupling.
- **Monitoring Server:** Tracks system performance, health, and resource usage for optimization and issue detection.
- **Registry Server:** Acts as a service registry, enabling microservices to discover and communicate with each other dynamically.

2. Proposed Technologies

- **Frontend:**
 - **Web App:** React.js or Angular (frontend); Node.js with Express or ASP.NET Core (backend).
 - **Mobile App:** Flutter or React Native (cross-platform); Swift (iOS) and Kotlin (Android) for native development.
- **API Gateway:** NGINX, Kong, or AWS API Gateway.
- **Backend Services:**
 - **Authentication & Authorization:** OAuth 2.0 with JWT; ASP.NET Core Identity or Keycloak.
 - **Room Management & Booking:** RESTful APIs with ASP.NET Core, Spring Boot, or Node.js with Express.
 - **Payment:** Stripe, PayPal, or Square integration.
 - **Notification:** SendGrid (email), Twilio (SMS), Firebase Cloud Messaging (push).
- **Load Balancer:** HAProxy, NGINX, or AWS Elastic Load Balancer.
- **Firewall (DMZ):** Cisco ASA, pfSense, or cloud-based firewalls.
- **Database:** PostgreSQL or MySQL (relational); MongoDB (NoSQL).
- **Monitoring Server:** Prometheus (monitoring), Grafana (visualization), ELK stack (logging).
- **Registry Server:** Eureka, Consul, or ZooKeeper.

3. Component Interactions

- **Client Requests:** Users access the system via the web or mobile app, sending requests to the **API Gateway**.
- **API Gateway:** Authenticates requests using the **Authentication Service**, then routes them to the relevant microservice (e.g., **Booking Service** for reservations).
- **Microservices:** Communicate via RESTful APIs or message queues (e.g., RabbitMQ). The **Registry Server** enables dynamic service discovery.
 - Example: A booking request goes to the **Booking Service**, which checks availability with the **Room Management Service**, processes payment via the **Payment Service**, and triggers a confirmation through the **Notification Service**.
- **Load Balancer:** Distributes traffic across multiple instances of the API Gateway and microservices for scalability.
- **Database:** Each microservice interacts with its own database (e.g., **Booking Service** updates its reservation records).
- **Monitoring:** The **Monitoring Server** collects metrics from all components to ensure system health.
- **Security:** The **Firewall** filters external traffic, while the **Authentication Service** enforces access control.

Continue:

Frontend

Web App

- **Technologies:**
 - **React.js or Angular** (frontend)
 - **Node.js with Express or ASP.NET Core** (backend)
- **Purpose:**
 - **React.js/Angular:** These JavaScript frameworks are ideal for creating dynamic and responsive user interfaces. They support efficient rendering, state management, and reusable components, ensuring a seamless

experience for users browsing hotels, checking availability, and managing bookings.

- **Node.js with Express:** A lightweight, non-blocking backend framework that handles server-side logic, such as serving static files, processing API requests, and managing user sessions. It's fast and suitable for I/O-heavy tasks.
- **ASP.NET Core:** A high-performance, cross-platform backend framework that offers robust server-side capabilities, strong integration with .NET tools, and scalability for handling web app requests.

Mobile App

- **Technologies:**
 - **Flutter or React Native** (cross-platform)
 - **Swift (iOS) and Kotlin (Android)** (native development)
- **Purpose:**
 - **Flutter/React Native:** These frameworks allow developers to write a single codebase that runs on both iOS and Android, reducing development time and costs while delivering near-native performance for mobile hotel booking features.
 - **Swift:** Apple's native language for iOS, providing top performance, access to iOS-specific features, and a polished user experience tailored to iPhone and iPad users.
 - **Kotlin:** Google's preferred language for Android, offering high performance, interoperability with Java, and native access to Android features for a seamless mobile experience.

API Gateway

- **Technologies:**
 - **NGINX, Kong, or AWS API Gateway**
- **Purpose:**

- **NGINX:** A high-performance web server and reverse proxy that routes client requests to appropriate microservices, handles load balancing, and secures traffic with SSL termination.
 - **Kong:** An open-source API gateway built on NGINX, adding features like authentication, rate limiting, and logging to streamline API management.
 - **AWS API Gateway:** A fully managed AWS service that simplifies API deployment, offering traffic management, authorization, and monitoring for cloud-based systems.
-

Backend Services (Microservices)

Authentication & Authorization Service

- **Technologies:**
 - **OAuth 2.0 with JWT**
 - **ASP.NET Core Identity or Keycloak**
- **Purpose:**
 - **OAuth 2.0 with JWT:** OAuth 2.0 provides a standard for secure authorization, while JWT (JSON Web Tokens) enables token-based authentication, ensuring safe user login and access control across services.
 - **ASP.NET Core Identity:** A built-in system for ASP.NET Core, managing user authentication, authorization, and account features like registration and login.
 - **Keycloak:** An open-source identity management tool supporting OAuth 2.0, OpenID Connect, and SSO, ideal for secure, scalable user authentication.

Room Management & Booking Services

- **Technologies:**
 - **RESTful APIs with ASP.NET Core, Spring Boot, or Node.js with Express**
- **Purpose:**
 - **ASP.NET Core:** A scalable framework for building RESTful APIs, offering high performance, dependency injection, and easy database integration for managing rooms and bookings.

- **Spring Boot:** A Java-based framework that simplifies RESTful API development with built-in support for microservices, security, and data handling.
- **Node.js with Express:** A lightweight option for fast, flexible APIs, excelling in I/O-heavy tasks like real-time room availability checks and booking updates.

Payment Service

- **Technologies:**
 - **Stripe, PayPal, or Square integration**
- **Purpose:**
 - **Stripe:** A versatile payment platform with APIs for processing online payments, subscriptions, and refunds, supporting multiple currencies and methods.
 - **PayPal:** A globally recognized payment gateway allowing users to pay via PayPal accounts or cards, with straightforward integration and wide reach.
 - **Square:** A simple payment solution for online and in-person transactions, offering easy integration and flexibility for hotel payments.

Notification Service

- **Technologies:**
 - **SendGrid (email)**
 - **Twilio (SMS)**
 - **Firebase Cloud Messaging (push notifications)**
- **Purpose:**
 - **SendGrid:** A cloud-based service for sending transactional emails (e.g., booking confirmations, password resets) with reliable delivery and API support.
 - **Twilio:** A platform for sending SMS and voice messages, used for real-time booking alerts or one-time passwords (OTPs).
 - **Firebase Cloud Messaging (FCM):** A Google service for sending push notifications to mobile and web apps, keeping users updated on booking statuses or promotions.

Load Balancer

- **Technologies:**
 - **HAProxy, NGINX, or AWS Elastic Load Balancer**
- **Purpose:**
 - **HAProxy:** A high-performance load balancer that distributes traffic across servers, ensuring no single instance is overwhelmed and maintaining system reliability.
 - **NGINX:** Beyond its API gateway role, NGINX can balance traffic using algorithms like round-robin, enhancing scalability and availability.
 - **AWS Elastic Load Balancer (ELB):** A managed AWS service that automatically scales and distributes traffic across instances, ensuring high availability in cloud deployments.

Firewall (DMZ)

- **Technologies:**
 - **Cisco ASA, pfSense, or cloud-based firewalls (e.g., AWS WAF)**
- **Purpose:**
 - **Cisco ASA:** A hardware firewall providing advanced threat protection, VPN support, and network segmentation to secure the system.
 - **pfSense:** An open-source firewall with enterprise features like intrusion detection and traffic control, offering cost-effective security.
 - **AWS WAF:** A cloud-based web application firewall protecting against common threats like SQL injection and DDoS attacks.

Database

- **Technologies:**
 - **PostgreSQL or MySQL (relational)**

- **MongoDB** (NoSQL)
 - **Purpose:**
 - **PostgreSQL/MySQL:** Relational databases ensuring data integrity and supporting complex queries for structured data like bookings, user profiles, and room details.
 - **MongoDB:** A NoSQL database offering flexibility and scalability for unstructured data, such as logs or user-generated content.
-

Monitoring Server

- **Technologies:**
 - **Prometheus** (monitoring)
 - **Grafana** (visualization)
 - **ELK Stack** (logging)
 - **Purpose:**
 - **Prometheus:** A monitoring tool that collects and stores system metrics, enabling proactive alerting and performance tracking.
 - **Grafana:** A visualization platform that creates real-time dashboards from Prometheus data, helping monitor system health and usage.
 - **ELK Stack (Elasticsearch, Logstash, Kibana):** A logging suite where Logstash processes logs, Elasticsearch indexes them, and Kibana visualizes them for troubleshooting and analysis.
-

Registry Server

- **Technologies:**
 - **Eureka, Consul, or ZooKeeper**
- **Purpose:**
 - **Eureka:** A Netflix-developed service registry for microservices, enabling dynamic registration and discovery of services.

- **Consul:** A service mesh tool offering discovery, health checking, and configuration management for microservices.
- **ZooKeeper:** A distributed coordination service ensuring synchronization and configuration across microservices in a scalable system.